

ПРАКТИЧЕСКАЯ 7: РАБОТА С ВРЕМЕННЫМИ РЯДАМИ

Ожидаемое время выполнения: 2 часа (1 час — анализ и построение графиков, 1 час — оформление отчёта).

Цель практики

Научиться анализировать временные ряды, выявлять тренды и сезонность, а также использовать визуальные инструменты для их интерпретации в Python, R.

Задачи практики

1. Выполнить анализ временных рядов:

- Загрузить данные временных рядов в Python, R.
- Выявить тренды и сезонность.
- Python: использование pandas (rolling, resample).
- R: использование пакетов forecast, zoo.

2. Выполнить визуализацию временных рядов:

- Построение графиков трендов, сезонности и остатков:
- Python: matplotlib и seaborn.
- R: ggplot2 и forecast.

3. Сравнить результаты анализа и визуализации в Python, R

4. Определить, какие инструменты наиболее подходят для анализа временных рядов в зависимости от задач.

Формат отчета

1. Введение:

- Цель и задачи работы.
- Краткое описание временных рядов и их компонентов (тренд, сезонность, остатки).

2. Результаты работы:

- Графики временных рядов:

- Скриншоты из Python (графики трендов и сезонности).
- Скриншоты из R (графики временных рядов).
- Выводы по трендам и сезонным компонентам на основе анализа.

3. Заключение:

- Сравнительный анализ Python, R для анализа временных рядов.
- Рекомендации по выбору инструмента в зависимости от типа задачи.

Дополнительные материалы к практической работе 7

Исходные данные: Набор временных данных (например, ежемесячные продажи, еженедельные посещения сайта).

ПОШАГОВАЯ ИНСТРУКЦИЯ К ПРАКТИЧЕСКОЙ РАБОТЕ 7

ШАГ 1. ПОДГОТОВКА ДАННЫХ

1.1. Общие требования к данным

Тип данных: Набор временных данных с указанием времени (даты, недели, месяцы и т.д.) и соответствующих показателей (например, объёмы продаж, количество посещений, стоимости акций).

Формат: Желательно, чтобы временная метка была в формате, распознаваемом Python/R как дата/время (ISO-8601, YYYY-MM-DD и т.д.).

Очистка: Убедитесь, что нет пропущенных значений или выбросы обработаны корректно. При необходимости сгруппируйте данные по периоду (день, неделя, месяц) для удобства анализа.

1.2. Python (pandas, matplotlib, seaborn)

Импорт библиотек:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Если планируете использовать statsmodels для продвинутых методов, подключите и его:

```
# pip install statsmodels (если не установлено)
import statsmodels.api as sm
```

Загрузка данных:

```
df = pd.read_csv('time_series_data.csv', parse_dates=['Date'],
index_col='Date')
```

```
print(df.head())
```

Проверка:

```
print(df.info())  
print(df.describe())
```

Убедитесь, что столбец с датами распознан как `datetime64[ns]`.

1.3. R (forecast, zoo, ggplot2)

Установка и импорт необходимых пакетов (при необходимости):

```
# install.packages("forecast")  
# install.packages("zoo")  
# install.packages("ggplot2")  
library(forecast)  
library(zoo)  
library(ggplot2)
```

Загрузка данных:

```
# Предположим, что файл называется "time_series_data.csv" и  
# содержит столбцы Date, Value  
df <- read.csv("time_series_data.csv", stringsAsFactors = FALSE)  
head(df)
```

Преобразование формата даты:

```
df$Date <- as.Date(df$Date, format = "%Y-%m-%d") # если даты в  
# формате YYYY-MM-DD  
str(df)
```

Создание временного ряда (опционально):

```
# Предположим, что данные идут с дневным шагом  
# Можно создать объект ts (для forecast) или zoo/xts  
df_ts <- ts(df$Value, start=c(2020,1), frequency=365) # пример  
# для ежедневных данных
```

ШАГ 2. АНАЛИЗ ВРЕМЕННЫХ РЯДОВ

2.1. Теоретические аспекты

Компоненты временного ряда:

Тренд: общее направление изменения показателя (рост/снижение) на длинном промежутке.

Сезонность: регулярные колебания, повторяющиеся в одинаковые периоды (дни недели, месяцы года и т.д.).

Остатки (ошибки): часть, не объяснённая трендом и сезонностью.

Стационарность: для прогноза часто важен переход к стационарным рядами (убирают тренд и сезонность). Но на данном этапе основной упор — на выявление и визуализацию структуры ряда.

Ресемплинг и скользящие статистики:

Ресемплинг (resampling) позволяет агрегировать данные по нужному интервалу (неделя, месяц).

Скользящее среднее (rolling mean) или скользящая дисперсия (rolling std) помогают сгладить колебания и оценить тренды.

2.2. Python

Ресемплинг и расчёт скользящего среднего

```
# Ежемесячный ресемплинг (если данные изначально дневные):  
df_monthly = df.resample('M').mean()  
  
# Скользящее среднее с окном в 7 дней (если у вас ежедневные  
данные):  
df['rolling_mean_7'] = df['Value'].rolling(window=7).mean()
```

Визуальная оценка тренда и сезонности

```
plt.figure(figsize=(10, 5))  
plt.plot(df.index, df['Value'], label='Исходный ряд')  
plt.plot(df.index, df['rolling_mean_7'], label='7-дневное
```

```

скользящее среднее')
plt.title("Анализ временного ряда (тренд и сглаживание)")
plt.xlabel("Дата")
plt.ylabel("Значение")
plt.legend()
plt.show()

```

Декомпозиция временного ряда (statsmodels)

```

from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(df['Value'],
model='additive', period=30)
# period=30 для месячной сезонности (пример, если данные дневные)

trend = decomposition.trend
seasonal = decomposition.seasonal
residual = decomposition.resid

decomposition.plot()
plt.show()

```

2.3. R (forecast, zoo)

Создание объекта временного ряда

```

# df_ts уже создан выше (для ежедневных данных, к примеру)
df_ts <- ts(df$Value, start=c(2020,1), frequency=365)

```

Декомпозиция

```

decomp <- decompose(df_ts, type="additive") # или
type="multiplicative"
plot(decomp)

```

Пример с пакетом forecast

```

library(forecast)

# Автоматическая декомпозиция STL
fit <- stl(df_ts, s.window="periodic")
plot(fit)

```

ШАГ 3. ВИЗУАЛИЗАЦИЯ ВРЕМЕННЫХ РЯДОВ

3.1. Python (matplotlib, seaborn)

Базовый график:

```
plt.figure(figsize=(10, 5))
plt.plot(df.index, df['Value'], marker='o')
plt.title("Временной ряд")
plt.xlabel("Дата")
plt.ylabel("Значение")
plt.show()
```

Сезонность (пример с boxplot):

```
# Предположим, нужно показать сезонность по месяцам (для годового
цикла):
df['Month'] = df.index.month
sns.boxplot(x='Month', y='Value', data=df)
plt.title("Сезонность по месяцам")
plt.show()
```

3.2. R (ggplot2, forecast)

Базовый график:

```
ggplot(df, aes(x=Date, y=Value)) +
  geom_line() +
  ggtitle("Временной ряд") +
  xlab("Дата") + ylab("Значение")
```

Сезонность (boxplot по месяцам):

```
df$Month <- format(df$Date, "%m")
ggplot(df, aes(x=Month, y=Value)) +
  geom_boxplot() +
  ggtitle("Сезонность по месяцам") +
  xlab("Месяц") + ylab("Значение")
```

ШАГ 4. СРАВНЕНИЕ РЕЗУЛЬТАТОВ И ОФОРМЛЕНИЕ ВЫВОДОВ

Сравните тренды, выявленные в Python (через скользящие средние и `decomposition`), R (через `decompose()` или `stl()`).

Сравните сезонность:

Убедитесь, что периоды пиковой активности/спадов совпадают в разных инструментах.

Отметьте, какая визуализация кажется более удобной и информативной.

Интерпретация:

Опишите, что вы узнали о сезонном поведении ряда (например, большее значение в определённые месяцы года).

Укажите, есть ли явный тренд (растущий/убывающий).

КЛЮЧЕВЫЕ МОМЕНТЫ ДЛЯ УСПЕШНОГО ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

Корректность дат: убедитесь, что во всех инструментах дата распознана как временная величина, а не строка.

Сезонность: чётко определите период сезонности (дни в году, недели в месяце и т.д.), исходя из характера данных.

Визуализация: продемонстрируйте несколько графиков, отражающих разные аспекты ряда (исходный ряд, сглаженный, декомпозицию).

Сравнение: укажите отличия и сходства подходов в Python, R.

Данная пошаговая инструкция позволит выполнить анализ временных рядов, выявить тренды и сезонность, а также сравнить результаты и визуализации, полученные при работе с двумя разными инструментами.